

□ Prompt: Build "Celestia" — A Birth Sky Visualization App

Target Model: Claude Opus 5 **Stack:** Next.js 16 (App Router), TypeScript, Vanilla CSS, HTML5 Canvas / WebGL

Goal: An astronomically precise, visually transcendent birth sky experience

The Vision

Build a Next.js 15 application called "**Celestia**" that asks the user for their **date of birth**, **exact birth time**, and **birth location** (city/coordinates), then reveals a **photorealistic, astronomically accurate snapshot of the night sky at the exact moment they were born** — complete with precise moon phase, visible constellations, their zodiac sign, and planetary positions — delivered through a **cinematic reveal sequence** so polished it makes users say "what the f***."

This is not a toy horoscope app. This is an **experience**. Think Apple-level polish meets NASA-level accuracy meets
Interstellar-level atmosphere.

I. Application Architecture

```

app/
├─ layout.tsx           # Root layout with fonts, metadata, global providers
├─ page.tsx             # Landing + input experience (single page app)
├─ globals.css          # Design system, tokens, animations, all styles
├─ components/
│   ├─ StarField.tsx    # Infinite parallax star background (canvas)
│   ├─ BirthForm.tsx    # Date/time/location input with micro-interactions
│   ├─ CosmicReveal.tsx # The big reveal orchestrator component
│   ├─ NightSkyCanvas.tsx # Main WebGL/Canvas sky renderer
│   ├─ MoonRenderer.tsx # Precise moon phase renderer with earthshine
│   ├─ ConstellationMap.tsx # Zodiac constellation line-drawing with labels
│   ├─ ZodiacCard.tsx   # The sign reveal card with description
│   ├─ PlanetaryPositions.tsx # Where planets were at birth
│   ├─ ShootingStar.tsx # Random shooting star animations
│   └─ AudioController.tsx # Ambient sound controller (optional, toggle)
├─ lib/
│   ├─ astronomy.ts     # All astronomical calculation engines
│   ├─ moonPhase.ts     # Precise lunar phase calculator
│   ├─ starCatalog.ts   # Bright star database (Hipparcos subset)
│   ├─ constellations.ts # Constellation line data (IAU)
│   ├─ zodiac.ts        # Zodiac sign determination + metadata
│   └─ coordinates.ts   # Coordinate transformations (RA/Dec → Alt/Az)
└─ data/
    ├─ stars.json       # Top ~300 brightest stars with RA/Dec/magnitude
    ├─ constellations.json # Constellation stick-figure line segments
    └─ zodiacInfo.json   # Rich zodiac sign descriptions + metadata

```

II. The User Flow — Beat by Beat

Beat 1: The Landing (0–3 seconds)

The page loads into **darkness**. Pure black. Then:

- Frame 0–500ms:** A single, tiny star fades in at the center. Then another. Then a cascade — hundreds of stars bloom outward from center like the Big Bang, settling into a gentle, slowly drifting star field with subtle parallax on mouse movement.
- Frame 500ms–1.5s:** The title "CELESTIA" fades in — not all at once, but letter by letter with a soft luminous glow, using a refined serif font (like *Cormorant Garamond* or *Playfair Display*). Each letter has a very subtle shimmer, like starlight reflecting off metal. Below it, a thin line of light extends horizontally, like a horizon.

3. **Frame 1.5s–3s:** The subtitle appears with a typewriter-style reveal: *"The sky remembers the moment you arrived."* — in a delicate sans-serif (*Inter* or *Geist*), slightly translucent, tracking wide.
4. **Persistent:** Behind everything, the star field breathes — stars twinkle with randomized sinusoidal opacity, and every 15–30 seconds a shooting star streaks across at a random angle. Mouse movement causes subtle parallax drift (3 layers of stars at different depths/speeds).

Beat 2: The Input Form (User Interaction)

After the landing settles, a gentle **scroll indicator** pulses at the bottom — a thin chevron breathing up and down. On scroll (or click), the content transitions:

The form **doesn't look like a form**. It looks like an ancient celestial instrument. Design it as:

- **Date of Birth:** Three rotating drum-style selectors (day / month / year) with inertial scrolling, styled as translucent glass drums with engraved numerals. Alternatively, a beautifully styled calendar picker with constellation watermarks. When the user selects a date, a faint ripple of light emanates from the selection.
- **Birth Time:** An analog clock face that the user drags the hands on — hour and minute hands with smooth rotation. The clock face has zodiac symbols around the rim instead of numbers. As hands move, the background sky subtly shifts color temperature (dawn colors for morning times, deep navy for night, golden hour for evening).
- **Birth Location:** A search input with autocomplete for cities. When a city is selected, subtle coordinate numbers (lat/long) animate in below it in a monospace font, like a NASA readout. Use a free geocoding API or a hardcoded list of major cities with coordinates.
- **The "Reveal My Sky" Button:** Not a regular button. A circular element, like a telescope lens or an iris aperture. On hover, the aperture blades shift slightly. On click, the iris closes with a mechanical animation, then everything goes to black for the transition.

Beat 3: The Cosmic Reveal (The Money Shot — 3–8 seconds)

This is where you make people's jaws drop. After the iris closes:

1. **Phase 1 — The Void (0–1s):** Total darkness. A low, subsonic vibration feeling (achieved with a very subtle CSS scale pulse on the entire viewport — $1.0 \rightarrow 1.002 \rightarrow 1.0$, like a cosmic heartbeat).
2. **Phase 2 — The Zoom (1–3s):** The camera appears to be hurtling through space. Stars streak past like entering hyperspace (particle system with elongated star trails moving from center outward). This accelerates, the density of streaking stars increases, then suddenly —
3. **Phase 3 — The Snap (3–3.5s):** Everything STOPS. Hard cut. The precise night sky of their birth snaps into place. But it's not fully revealed yet — it starts as a blurred, out-of-focus bokeh of lights, then racks focus to sharp over 500ms (CSS blur filter transition from `blur(20px)` to `blur(0)`).

4. **Phase 4 — The Constellation Draw (3.5–6s)**: Their zodiac constellation draws itself on the sky — lines connecting stars one by one with a glowing trace animation (like a laser drawing in the air). Each line leaves a faint persistent glow. When the constellation is complete, its name writes itself next to it in elegant small-caps.
5. **Phase 5 — The Moon (5–7s)**: The moon rises from the bottom edge of the screen to its calculated position — rendered with **precise phase accuracy** (more on this below). As it rises, it casts a gentle, realistic bloom/glow that softly illuminates the surrounding area of the sky. If the moon was below the horizon at birth time, show it as a translucent ghost-moon at its sub-horizon position with text noting "The moon was below your horizon."
6. **Phase 6 — The Card (7–8s)**: A frosted-glass card materializes at the bottom of the screen (or as an overlay panel) with their zodiac sign symbol, name, date range, element (fire/water/earth/air with corresponding subtle color tint), and a one-paragraph personality description. The card has a glassmorphism style with backdrop-blur, subtle border luminance, and content that fades in line by line.

Beat 4: The Exploration (Post-Reveal)

After the reveal, the sky becomes **interactive**:

- **Pan and rotate**: User can click-drag to pan around the sky sphere (or at minimum, $\pm 60^\circ$ horizontal pan).
- **Star identification**: Hovering over bright stars shows a tooltip with the star's name (e.g., "Sirius — α Canis Majoris — Magnitude -1.46").
- **Constellation toggle**: A subtle toggle in the corner to show/hide all visible constellation lines and labels.
- **Planetary positions**: Small, distinctly colored dots for visible planets (Mercury, Venus, Mars, Jupiter, Saturn) at their calculated positions, with labels on hover.
- **Info panel**: A slide-out panel (from right) with detailed birth chart info:
 - Sun sign, Moon sign, Rising sign (if location provided)
 - Moon phase name and illumination percentage
 - Planetary positions in zodiac signs
 - A beautiful summary paragraph

III. Astronomical Precision — The Math

This is NON-NEGOTIABLE. The sky must be **correct**. Implement these calculations in `lib/astronomy.ts`:

A. Julian Date Conversion

Convert the birth date/time (in UTC) to Julian Date (JD). This is the foundation of all astronomical calculations.

$$JD = 367Y - \text{INT}(7(Y + \text{INT}((M+9)/12))/4) + \text{INT}(275M/9) + D + 1721013.5 + UT/24$$

B. Sun Position (Zodiac Sign Determination)

Calculate the **ecliptic longitude** of the Sun at the given JD using the low-precision solar position algorithm:

- Mean anomaly: $M = 357.5291 + 0.98560028 \times (JD - 2451545.0)$
- Equation of center: $C = 1.9148 \sin(M) + 0.02 \sin(2M) + 0.0003 \sin(3M)$
- Ecliptic longitude: $\lambda = M + C + 180 + 102.9372$
- The zodiac sign is determined by which 30° segment λ falls in (0°=Aries, 30°=Taurus, etc.)

C. Moon Phase Calculation

Calculate the **precise lunar phase angle** and **illumination fraction**:

- Use the moon's mean elongation, mean anomaly, and argument of latitude
- Calculate the phase angle (angle between Sun-Moon-Earth)
- Illumination fraction: $k = (1 + \cos(i)) / 2$ where i is the phase angle
- Determine phase name: New Moon, Waxing Crescent, First Quarter, Waxing Gibbous, Full Moon, Waning Gibbous, Last Quarter, Waning Crescent
- **Critical for rendering**: Calculate whether the lit side is on the left or right (waxing = right lit, waning = left lit in Northern Hemisphere — flip for Southern)

D. Moon Position

Calculate the Moon's RA/Dec using simplified lunar theory:

- Mean longitude, mean anomaly, mean elongation
- Apply principal perturbation terms (at minimum the evection, variation, and annual equation)
- Convert ecliptic coordinates to equatorial (RA/Dec)

E. Sidereal Time and Coordinate Transformation

To render the sky correctly for a **specific location and time**:

- Calculate Greenwich Mean Sidereal Time (GMST) from JD
- Calculate Local Sidereal Time (LST) = GMST + observer longitude
- Transform star positions from RA/Dec (catalog) to Altitude/Azimuth (observer):
 - Hour Angle: $HA = LST - RA$
 - $Alt = \arcsin(\sin(Dec) \sin(Lat) + \cos(Dec) \cos(Lat) \cos(HA))$
 - $Az = \arctan2(-\sin(HA) \cos(Dec), \cos(Dec) \sin(Lat) \cos(HA) - \sin(Dec) \cos(Lat))$
- Only render stars with Altitude > 0° (above horizon)

F. Planetary Positions (Simplified)

For the 5 visible planets, use simplified orbital elements (mean longitude, perihelion, eccentricity, inclination) to calculate approximate ecliptic positions. Convert to RA/Dec, then to Alt/Az for the observer. Accuracy to ±1° is sufficient.

G. Star Catalog

Embed a JSON catalog of the **~300 brightest stars** (down to about magnitude +4.0 for naked-eye visibility). Each entry needs:

```
{
  "name": "Sirius",
  "bayer": "α CMa",
  "ra": 101.287,      // degrees
  "dec": -16.716,    // degrees
  "mag": -1.46,
  "constellation": "CMa",
  "color": "#A8C4F0"  // spectral color approximation
}
```

Star colors should map from spectral type: O/B = blue-white, A = white, F = yellow-white, G = yellow, K = orange, M = red.

H. Constellation Data

Embed constellation stick-figure data as pairs of star indices for the **zodiac constellations** at minimum (all 12), plus major recognizable ones (Orion, Ursa Major, Cassiopeia, Crux, etc. — at least 30 total). Format:

```
{
  "name": "Orion",
  "abbr": "Ori",
  "lines": [[star1_index, star2_index], [star2_index, star3_index], ...]
}
```

IV. Visual Rendering Specifications

Night Sky Canvas (`NightSkyCanvas.tsx`)

Use an **HTML5 Canvas** (2D context is fine; WebGL if you're feeling ambitious) that fills the viewport.

Stars:

- Render each visible star as a radial gradient dot — brighter stars are larger (map magnitude to radius: $r = \max(1, 4 - \text{mag} * 0.8)$ pixels).
- Star colors should be accurate to spectral type (use the color from catalog).
- Each star has a subtle twinkle: a sinusoidal opacity oscillation with a randomized frequency (0.5–2Hz) and phase offset. Bright stars ($\text{mag} < 1$) should also have a very faint 4-point diffraction spike effect (two thin intersecting lines through the star center, very low opacity).

- Apply a subtle Gaussian glow to stars brighter than magnitude 1 (render a larger, very transparent circle behind them).

Sky gradient:

- The background is NOT flat black. It's a subtle radial gradient simulating light pollution / atmospheric glow near the horizon:
 - Zenith: #0a0a1a (near black with slight blue)
 - 30° altitude: #0d0d24
 - Horizon: #141428 (slightly lighter, hinting at atmospheric scattering)
- If birth time is during twilight (Sun altitude between -18° and 0°), blend in appropriate twilight colors (deep orange at horizon fading to navy).

Milky Way (bonus, but incredible if done):

- Render a very subtle, cloudy band of increased star density / faint nebulosity along the galactic plane. This can be a pre-rendered semi-transparent PNG overlay positioned based on galactic coordinates, or a procedural noise band. Keep it subtle — it should feel like you *almost* can't see it, like a real dark sky.

Moon Renderer (MoonRenderer.tsx)

The moon is THE showpiece visual. Render it large (80–120px diameter) and beautiful:

1. **Base disk:** A warm gray circle (#C8C0B0) with subtle surface texture (use procedural noise or a very subtle radial variation to suggest maria/highlands — you can overlay semi-transparent darker ellipses for major maria like Mare Tranquillitatis, Mare Serenitatis, etc.).
2. **Phase shadow:** Apply the terminator (day/night boundary on the moon) as a clipping mask or overlay:
 - The terminator is an **ellipse** whose semi-minor axis equals $\cos(\text{phase_angle}) \times \text{radius}$
 - For waxing phases: shadow on the left, lit on the right
 - For waning phases: shadow on the right, lit on the left
 - The shadow side should not be pure black — render it as very dark gray (#1a1a22) with a faint bluish tint to simulate **earthshine** (sunlight reflected off Earth illuminating the dark side). This earthshine should show a ghostly hint of the surface features.
3. **Glow:** Behind the moon, render a multi-layered glow:
 - Inner glow: 1.3× radius, warm white, opacity 0.15
 - Outer glow: 2.5× radius, cool white, opacity 0.05
 - The glow intensity should scale with illumination fraction
4. **Limb darkening:** The edges of the lit portion should be very slightly darker than the center (multiply with a radial gradient that's 1.0 at center, 0.85 at edge).

Horizon Line

Render a very subtle horizon line at the bottom of the canvas — not a hard line, but a gradient fade from sky color to slightly lighter, with perhaps a silhouette of a gentle hill/mountain line (a simple bezier curve, very dark, almost imperceptible).

V. Animation & Transition Specifications

All animations should use **CSS custom easing curves** for a premium feel. No linear or basic ease-in-out. Use curves like:

```
--ease-out-expo: cubic-bezier(0.16, 1, 0.3, 1);  
--ease-out-quart: cubic-bezier(0.25, 1, 0.5, 1);  
--ease-in-out-quint: cubic-bezier(0.83, 0, 0.17, 1);  
--spring: cubic-bezier(0.34, 1.56, 0.64, 1); /* overshoot */
```

Specific Animation Details:

Star twinkle: `opacity: 0.6 + 0.4 * sin(t * frequency + phase)` — run on canvas, not CSS.

Title letter reveal: Each letter delays by 60ms. Opacity 0→1 over 400ms + `translateY(10px→0)` simultaneously. After all letters land, a brief flash of brightness (opacity 1→1.3→1 via filter brightness).

Constellation line draw: Each line segment draws over 300ms using a `lineDash + lineDashOffset` animation pattern on canvas, or by interpolating the line endpoint. The line should have a "glowing tip" — a brighter, slightly larger dot at the drawing endpoint that dims after the line completes.

Iris aperture button: 6 triangular blade shapes arranged radially, rotating inward on click. Use CSS clip-path or SVG for the blades. The close animation should take 600ms with a mechanical, satisfying feel.

Hyperspace zoom: Particle system on canvas — 500+ particles spawning at center with randomized radial velocities. Each particle draws as a line from its current position to its position 2 frames ago (motion blur trail). Acceleration increases over 2 seconds. White with random slight blue/warm tints.

Glassmorphism card entrance: `translateY(40px)→0, opacity 0→1, backdrop-filter: blur(0px)→blur(20px), scale(0.95)→scale(1)` — all over 800ms with `--ease-out-expo`.

VI. Design System & Typography

Color Palette

```
--void:          #050510;      /* Deepest background */
--cosmos:        #0a0a1a;      /* Primary background */
--nebula:        #141428;      /* Elevated surfaces */
--stardust:      #1e1e3a;      /* Cards, panels */
--starlight:     #e8e4f0;      /* Primary text */
--moonbeam:      #b8b0c8;      /* Secondary text */
--celestial:     #8888cc;      /* Accents, interactive */
--nova:          #c8a050;      /* Gold accent — zodiac symbols */
--ember:         #e85040;      /* Fire signs */
--ocean:         #4088d0;      /* Water signs */
--terra:         #60a848;      /* Earth signs */
--zephyr:        #c8c040;      /* Air signs */
--glass-bg:      rgba(20, 20, 50, 0.6);
--glass-border:  rgba(255, 255, 255, 0.08);
```

Typography

- **Display/Title:** `Cormorant Garamond` (serif, elegant, celestial feel) — for "CELESTIA", zodiac names, and headings. Use 300/400 weight, generous letter-spacing (0.2em for titles).
- **Body:** `Inter` or `Geist Sans` — for descriptions, UI text. Use 300/400 weight, line-height 1.7.
- **Monospace accent:** `JetBrains Mono` or `Geist Mono` — for coordinates, dates, technical readouts. Use at small sizes with wide letter-spacing.
- Load from Google Fonts (`Cormorant Garamond` + `Inter`) or Next.js built-in (`next/font`).

Spacing & Layout

- Use a base unit of 8px. All spacing should be multiples: 8, 16, 24, 32, 48, 64, 96.
- Maximum content width: 1200px, centered.
- The sky canvas is ALWAYS fullscreen (`100vw × 100vh, position: fixed`).
- UI elements float above the canvas with `position: relative` or `absolute` and high `z-index`.

VII. Zodiac Data

Each zodiac sign needs rich data:

```
interface ZodiacSign {
  name: string;           // "Aries"
  symbol: string;         // "♈"
  unicode: string;        // The actual unicode symbol
  dateRange: string;      // "March 21 - April 19"
  element: "fire" | "water" | "earth" | "air";
  modality: "cardinal" | "fixed" | "mutable";
  rulingPlanet: string;   // "Mars"
  traits: string[];       // ["courageous", "determined", "confident", "passionate"]
  description: string;    // 2-3 sentence poetic description
  constellation: string;  // Reference to constellation data
  mythology: string;      // 1-2 sentence mythological origin
}
```

Write genuinely compelling, poetic descriptions — not generic horoscope fluff. Example for Pisces:

"Born under the sign of the Fish, you carry the depth of every ocean within you. Pisces is the final sign — the culmination of every lesson the zodiac has learned — granting you an intuition that borders on the otherworldly. You feel what others cannot name, and dream in colors that don't yet exist."

VIII. Responsive Design

- **Desktop (1024px+):** Full experience as described. Sky is fullscreen. Form and cards are overlaid.
- **Tablet (768–1023px):** Same experience, slightly smaller moon and card. Touch interactions replace hover.
- **Mobile (< 768px):**
 - Star field still renders fullscreen on canvas (reduce star count to 150 for performance).
 - Form becomes full-width stacked inputs (keep the aesthetic, adapt the layout).
 - The clock input becomes a standard time picker, beautifully styled.
 - The reveal card becomes a full-screen overlay with scroll.
 - Touch-drag for sky panning.
 - All font sizes scale down proportionally.

IX. Performance Considerations

- **Canvas rendering:** Use `requestAnimationFrame` for the star field. Implement frame-rate limiting (cap at 60fps). Cache static elements that don't change per frame.
- **Star catalog:** Load the star JSON lazily (dynamic import) — only needed after form submission.
- **Calculations:** All astronomy math runs on the client. It's pure math and will be fast. No API calls needed.

- **Images:** No external images. Everything is rendered procedurally (canvas, CSS, SVG).
 - **Bundle size:** Keep it lean. No heavy animation libraries unless truly needed. Prefer CSS animations and canvas over DOM-heavy solutions. If you use Framer Motion, be surgical — only for the UI element transitions, not for the sky rendering.
-

X. Key Technical Constraints

1. **No external APIs for astronomy.** All calculations must be self-contained in the codebase.
 2. **No placeholder content.** Every star, every constellation line, every zodiac description must be real data.
 3. **The star catalog must have real astronomical coordinates** (Right Ascension and Declination in degrees, real magnitudes, real names). Use data from the Hipparcos catalog or Yale Bright Star Catalog.
 4. **Moon phase must be visually correct for the given date.** If someone enters July 20, 1969 (Apollo 11 landing), the moon should show a waxing crescent (which it was).
 5. **Zodiac sign determination must use the actual ecliptic longitude of the Sun**, not just date ranges (which can be off by a day depending on the year).
 6. **The app must work entirely client-side** after the initial page load. No server-side API routes needed for core functionality.
-

XI. Bonus Features (Implement if Possible)

1. **"Share Your Sky" button:** Generates a beautiful OG-image-ready screenshot of the sky (using canvas `.toDataURL()`) with the user's name, date, and sign overlaid — ready to download or share.
 2. **Ambient sound toggle:** A small speaker icon in the corner. When enabled, plays a subtle ambient drone/pad sound (use the Web Audio API to generate a soft, evolving pad with oscillators — no audio files needed). Very low volume, ethereal.
 3. **Zodiac sign transition:** If the user's birthday is within 2 days of a sign cusp, show a subtle note: *"Born on the cusp of [Sign1] and [Sign2]"* with both constellations subtly highlighted.
 4. **Compatibility teaser:** After revealing the sky, a subtle prompt: *"Enter another birthday to compare your skies"* — which overlays a second person's constellation in a different color.
 5. **Current sky comparison:** A toggle to flip between "Your Birth Sky" and "Tonight's Sky" with a smooth crossfade transition showing how the sky has changed.
-

XII. What "Done" Looks Like

When this app is complete, a user should be able to:

1. Land on the page and feel **immediately immersed** — like opening a portal to the cosmos
2. Enter their birth details and feel like they're **programming a celestial machine**
3. Hit the reveal button and experience a **6-second cinematic sequence** that makes them hold their breath
4. See their **actual birth sky** — accurate moon, real constellations, correct zodiac sign
5. Explore the sky, learn about their sign, and feel a genuine moment of **cosmic wonder**
6. Want to **immediately share it** with someone and make them try it too

The quality bar is: **"I can't believe this runs in a browser."**

XIII. Final Implementation Notes

- Use Next.js 15 with the App Router. This can be a single-page experience (just `app/page.tsx` as the main route).
- Use `"use client"` for all interactive components.
- Implement everything in a single project — no microservices, no external dependencies beyond Next.js core and possibly Framer Motion.
- CSS should be in `globals.css` using CSS custom properties for the design system. No Tailwind.
- Test with these known dates to verify accuracy:
 - **July 20, 1969** — Sun in Cancer, Moon waxing crescent (~day 6)
 - **December 25, 2000** — Sun in Capricorn
 - **March 20, 1990** — Sun at Pisces/Aries cusp
 - **January 1, 2000, 00:00 UTC** — A common reference epoch

Now build it. Make it extraordinary. ✦